

DOSR: Decentralized Open-Source Review Project Checkpoint

William Convertino
wac20

Jakub Romanczuk
jr614

Gage Garcia
gv7

Yuhan Wei
yw641

1 Introduction

We propose **DOSR** (Decentralized Open-Source Review), a system for maintaining peer-to-peer software repositories whose proposed changes are approved using cryptographically attested LLM reviews. Contributors pay for the evaluation of their own changes, avoiding a costly centralized evaluation, while a distributed consensus layer maintains the canonical repository state. Unlike conventional repository hosting, DOSR supports completely decentralized repositories without any privileged maintainers or persistent users, making it possible for transient human or autonomous agents to contribute to a shared project under a fixed review policy.

2 High-Level Design

Figure 1 shows an overview of the DOSR protocol. DOSR has three main components: Git and a peer-to-peer storage layer hold and distribute repository contents, an LLM and attestation service determine whether a proposed change satisfies the repository’s review policy, and an Ethereum Virtual Machine (EVM) [3] compatible blockchain maintains the canonical repository state. This keeps large data and expensive model inference off-chain while still giving all participants a common, independently verifiable view of which commit is current.

0. Create a repository. A DOSR repository begins as a normal Git repository with the initial code files and a DOSR configuration file. The configuration defines the rules under which future changes may be accepted, including which LLM providers, models, and model versions can be used for PRs, the format of the PR reviews, and the public keys of trusted attestation authorities. The initial Git commit and a hash of this configuration are then registered with the DOSR smart contract.

The review policy is intentionally fixed for the lifetime of the repository and cannot be modified through an ordinary PR. Otherwise, a contributor could propose a change that replaces the approved models or attestation keys and effectively take control of the repository. Users who want a different re-

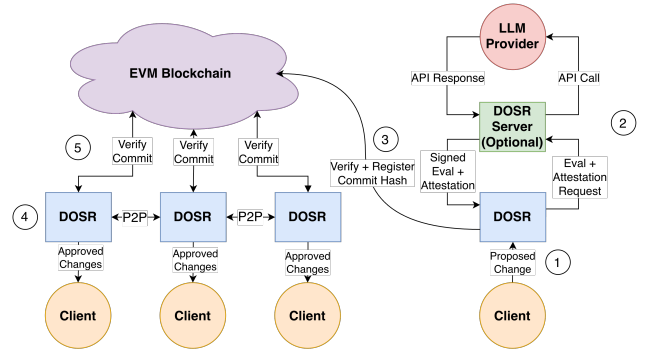


Figure 1: An overview of the DOSR protocol. A repository is initialized with a fixed review policy (0). A contributor proposes a change (1) and obtains an attested LLM review (2). If accepted, the contributor registers the commit on an EVM-compatible blockchain (3). Repository contents are replicated separately between peers (4), and other users retrieve and verify the canonical state against the blockchain (5).

view policy instead must create a fork with any configuration changes.

1. Propose a change.

To contribute to the repository, a contributor first makes their changes locally and creates a candidate Git commit, ensuring that the parent commit is the current canonical commit.

The proposed transition is packaged for review according to the repository’s configured PR format. This format specifies what information is sent to the model, such as changed files, surrounding code, repository context, and review instructions. DOSR provides some standard templates for common repository structures while also allowing repositories to define their own policies.

2. Evaluate and attest the PR. The proposed change is evaluated by an approved LLM, following the repository’s configuration. The contributor pays for this inference using their own API credentials, so the cost of reviewing a change is paid exclusively by the contributor rather than by a permanent

repository operator or other nodes on the network.

To avoid a user receiving LLM approval with an unapproved PR format, or otherwise tampering with the api call, we additionally provide a DOSR attestation service. Given a proposed PR, the attester verifies that the PR format and model used align with the policy of the repository. It then calls the selected model, using the clients provided API key, and signs an attestation over the resulting evaluation.

This attestation binds the review to the repository, parent commit, candidate commit, model information, and review result, along with hashes of the relevant request and response. With the signed attestation, one can provably see the attester’s verification that i. the PR followed the format detailed in the repository’s configuration, ii. the PR follows directly from the previous canonical head, iii. the model used was an approved model, again following the repository’s configuration, and iv. the LLM approved the PR.

We opted for this server-based attestation approach to avoid two major issues. First, if we verified the PR within the client application, an attacker could modify the application to accept either fake LLM approvals or real approvals for PRs that did not follow the repositories configuration. Alternatively, we could verify the commit within our EVM, ensuring that only valid commits are added to the blockchain. While this would make it effectively impossible to fake a PR acceptance, the transaction cost would be prohibitively expensive, as each EVM verifier would need to process the entire PR.

Our initial prototype will make use of an official DOSR attestation server, while the same interface could later support approved third-party attestors, such as TEE-backed services or native signatures from model providers. The attester used, and their corresponding public key, will be options that can be set in the DOSR configuration file.

3. Register an approved commit. If the model rejects the proposed change, the repository state remains unchanged. The contributor can revise the change and submit it for another review without interacting with the blockchain.

If the change is accepted, the contributor submits the candidate commit and signed attestation to the DOSR smart contract and pays the transaction fee. The contract checks that the attestation was signed by an approved authority, that the model and review satisfy the repository policy, and that the attested parent commit is still the current canonical HEAD. If these checks succeed, the candidate commit becomes the new canonical head.

DOSR deliberately keeps this on-chain state small. The blockchain does not store source code, Git objects, complete prompts, or LLM responses, instead leaving those to the P2P network. Instead, the blockchain stores only the information needed to identify and validate accepted repository transitions.

It is possible for multiple contributors to receive valid approvals based on the same parent commit. In that case, whichever accepted update is registered first becomes the new canonical HEAD. Any later proposal based on the old parent

fails the contract’s parent check and must be rebased and reviewed again.

4. Replicate the accepted repository state. The blockchain identifies the canonical commit but does not store its files. The underlying repository remains a Git repository, so commits, trees, and blobs retain Git’s existing content-addressed representation. This follows the approach of many distributed repositories [4, 11].

For the prototype, DOSR will use IPFS [2], an existing peer-to-peer content distribution system, to replicate repository data between clients. The peer-to-peer storage identifier is used to locate and distribute the data, but ultimately the canonical state is verified by the EVM.

5. Read and verify the canonical state. When a user wishes to see the most up-to-date version of a DOSR repository, they first query the EVM to determine the current canonical HEAD. Viewing the blockchain does not have any associated cost.

The user can then retrieve the corresponding Git data from any available peer or mirror and verify that it contains the commit recorded by the contract. Peers therefore do not have to be trusted to tell a user which version of the repository is current. The peer-to-peer network provides the data, while the blockchain provides the authoritative reference against which that data is checked.

Advantages of DOSR.

DOSR provides several advantages over traditional repository hosting. By distributing repository storage and maintaining canonical state through a blockchain, repositories can persist without relying on a centralized hosting provider or privileged maintainers. This also allows for transient contributors, including autonomous agents, to participate without needing to create and manage an account, and avoids repositories from freezing if the agent that created it is shut down. Furthermore, the cost of submitting a PR is kept low and entirely centered on the submitter, allowing view-only clients to use the system entirely for free.

3 Related Work

AI PR Review. AI driven PR reviews has become a prevalent topic in both academia and industry. Systems such as Qodo/PR-Agent and Mira use LLMs to analyze proposed code changes and provide review feedback [7, 10]. BitsAI-CR combines rule-guided issue detection with a separate review-filtering stage [13], while RovoDev uses structured prompting, repository context, and checks for factual correctness and actionability [14]. Despite these advances, reliably evaluating LLM-generated review feedback remains an open problem [1, 12].

DOSR does not attempt to solve the underlying problem of making LLM code review perfectly reliable, but this research fundamentally underpins the efficacy of our system. As models and techniques for automated PR review improve, DOSR

will become more reliable and robust.

Verifiable LLM Calls. Several systems provide mechanisms for authenticating externally generated data or LLM responses. TLSNotary allows a user to produce cryptographic evidence of data exchanged during a TLS session [15]. Open-Gradient instead routes LLM requests through a trusted execution environment and signs hashes of the request and response [9]. These approaches motivate DOSR’s use of an attestation layer between LLM evaluation and repository consensus. Our initial implementation uses a DOSR-operated attestation service, while the protocol is designed to allow other trusted attestors or provider-native signatures in the future.

Decentralized Repositories. Several systems explore decentralized source-code management. Radicle [11] provides peer-to-peer Git replication and uses cryptographically signed repository state and delegate-based policies to coordinate repositories. Gitopia [4] uses blockchain-backed metadata together with decentralized storage, while GOSH [5] supports blockchain-based repositories and DAO voting for protected repository operations. BDA-SCV [6] and other work [8] similarly explore blockchain-based version control. DOSR differs from these systems in how proposed changes are authorized: rather than requiring a maintainer or governance vote to judge each PR, DOSR uses an attested LLM review and uses the consensus layer only to validate and order approved repository transitions.

4 Threat Analysis

Our preliminary work has involved analyzing and addressing the ways an attacker could take advantage of our protocol, as detailed below.

Malicious contributors. A contributor may modify the DOSR client to bypass review, alter the PR sent to the LLM, or associate an approval with a different commit. Our original design considered using TLSNotary [15] to authenticate the LLM request and response, but this alone cannot guarantee that the reviewed request corresponds to the actual repository transition, and there is, in fact, no way to attest this on the local client.

Meanwhile, doing this verification on the client nodes would add significant friction (requiring POW, POS, or trusted clients) and leave the system more vulnerable to attackers. Verifying the PRs on the EVM itself effectively solves this problem, but becomes prohibitively expensive as the size of the requests grows. Our solution is to use an attestation service that independently verifies the PR format, parent and candidate commits, and selected model before signing the approval. The EVM accepts a transition only if this signature is valid.

Malicious or compromised authorities. DOSR assumes that the configured LLM provider and attestation authority are trusted. A malicious LLM provider could return an incorrect

approval, while a malicious or colluding attestor could sign a PR without correctly validating it. These attacks are outside the guarantees of our protocol, and one must assume that these services are well-intentioned and well-run. With that said, these established providers are much more likely to be trustworthy than individual clients, and repository creators can specify which providers and attestors to trust. Further, future versions of DOSR could reduce this assumption by using multiple independent attestors or TEE-backed verification [9] to forgo attestors altogether.

Low-Effort or Malicious Commits.

LLM driven PR reviews is far from perfect, and a motivated attacker will likely be able to succeed at pushing through a bad commit with enough time and effort. With that said, we have considered a number of safeguards to address this.

In the attestor, we have added a per-client rate-limit to deter attackers from continually submitting bad requests with the hope of pushing one through. We have also considered caching recent failed requests for a given repository to detect exact or highly similar PR requests. Additionally, the attestor will have a built-in jailbreak detection system to avoid accepting PRs with prompt injects and similar attacks.

The most powerful defense tool, however, lies in the configuration file. On a per-repository level, the creator can select the approved models, the number of model approvals needed for acceptance, and the exact format of the PR request. By adjusting these settings, the security requirements of a repository can be tuned for the needs of that project.

Finally, we plan to do an analysis of the success rate and cost of a malicious PR, as there may be an economic deterrent as well. A good PR is more likely to be accepted, and therefore likely affordable. A bad PR, meanwhile, may take hundreds or thousands of reviews to get accepted, increasing the monetary cost significantly.

Malicious storage peers. While peers may attempt to pass along incorrect repository data, the DOSR application is able to verify the content address by checking the blockchain’s canonical state, rejecting any stale or incorrect states.

5 Responsibilities

Will is responsible for developing the DOSR client applications, including the P2P-backed Git interface and verification of the canonical repository state through the EVM. Yuhan and Jakub will develop the attestation service responsible for validating review requests and producing signed approvals. Jakub and Gage will evaluate different PR prompting formats and LLM models, including both review quality and the monetary cost of performing a PR evaluation. Will and Yuhan will evaluate the end-to-end latency of the system, including submitting and approving a PR, registering and verifying the resulting commit on the blockchain, and propagating the updated repository state through the peer-to-peer network.

References

- [1] Fannar Steinn Aðalsteinsson, Björn Borgar Magnússon, Mislav Milicevic, Adam Nirving Davidsson, and Chih-Hong Cheng. Rethinking code review workflows with llm assistance: An empirical study. In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 488–497, 2025.
- [2] Juan Benet. Ipfs – content addressed, versioned, p2p file system, 2014.
- [3] Ethereum Foundation. Interacting with smart contracts. <https://ethereum.org/developers/docs/smart-contracts/interacting/>. Accessed 2026.
- [4] Gitopia. Storage. <https://docs.gitopia.com/storage/index.html>. Gitopia documentation. Accessed: 2026-09-09.
- [5] GOSH. Organizations: GOSH DAO and Soft Majority Voting. <https://docs.gosh.sh/on-chain-architecture/organizations-gosh-dao-and-smv/>. GOSH documentation. Accessed: 2026-09-09.
- [6] Muhammad Hammad, Jawaaid Iqbal, Ch Anwar ul Hassan, Saddam Hussain, Syed Sajid Ullah, Mueen Uddin, Urooj Ali Malik, Maha Abdelhaq, and Raed Alsaqour. Blockchain-based decentralized architecture for software version control. *Applied Sciences*, 13(5):3066, 2023.
- [7] miracodeai. Mira: Self-hosted AI code reviewer. <https://github.com/miracodeai/mira>. GitHub repository. Accessed: 2026-09-09.
- [8] Nishara Nizamuddin, Khaled Salah, Muhammad Ajmal Azad, Junaid Arshad, and M. H. Rehman. Decentralized document version control using ethereum blockchain and ipfs. *Computers & Electrical Engineering*, 76:183–197, 2019.
- [9] OpenGradient. TEE Gateway: TEE-secured inference node for third-party LLM inference requests. <https://github.com/OpenGradient/tee-gateway>. GitHub repository. Accessed: 2026-09-09.
- [10] Qodo. Qodo Merge (and open-source PR-Agent): Overview. <https://qodo-merge-docs.qodo.ai/index>. Accessed: 2026-09-09.
- [11] Radicle. Radicle Protocol Guide. <https://radicle.xyz/guides/protocol/>. Accessed: 2026-09-09.
- [12] Kexin Sun, Hongyu Kuang, Sebastian Baltes, Xin Zhou, He Zhang, Xiaoxing Ma, Guoping Rong, Dong Shao, and Christoph Treude. Does ai code review lead to code changes? a case study of github actions. *IEEE Transactions on Software Engineering*, 52(7):2111–2126, 2026.
- [13] Tao Sun, Jian Xu, Yuanpeng Li, Zhao Yan, Ge Zhang, Lintao Xie, Lu Geng, Zheng Wang, Yueyan Chen, Qin Lin, Wenbo Duan, and Kaixin Sui. BitsAI-CR: Automated code review via LLM in practice, 2025.
- [14] Kla Tantithamthavorn, Yaotian Zou, Andy Wong, Michael Gupta, Zhe Wang, Mike Buller, Ryan Jiang, Matthew Watson, Minwoo Jeong, Kun Chen, and Ming Wu. RovoDev Code Reviewer: A large-scale online evaluation of LLM-based code review automation at Atlassian, 2026.
- [15] TLSNotary. Notarization. <https://tlsnotary.org/docs/protocol/notarization/>. TLSNotary protocol documentation. Accessed: 2026-09-09.